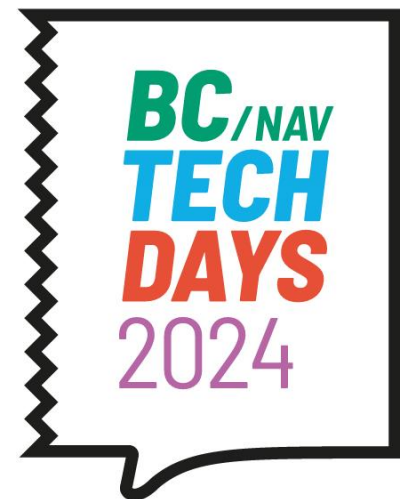




**Tine
Starič**



mibuso.com

Reviewing the Code Review

Tine Starič

Software Architect @ Companial



44 : 07 : 26 : 48

DAYS HOURS MINUTES SECONDS

Agenda

- Why do we do code review
- Review the process
- Review the reviewer
- Review the reviewee
- Are Copilots going to help?



Why Code Review?

- Ensuring code quality
- Following coding guidelines
- Find bugs and edge cases



Why Code Review

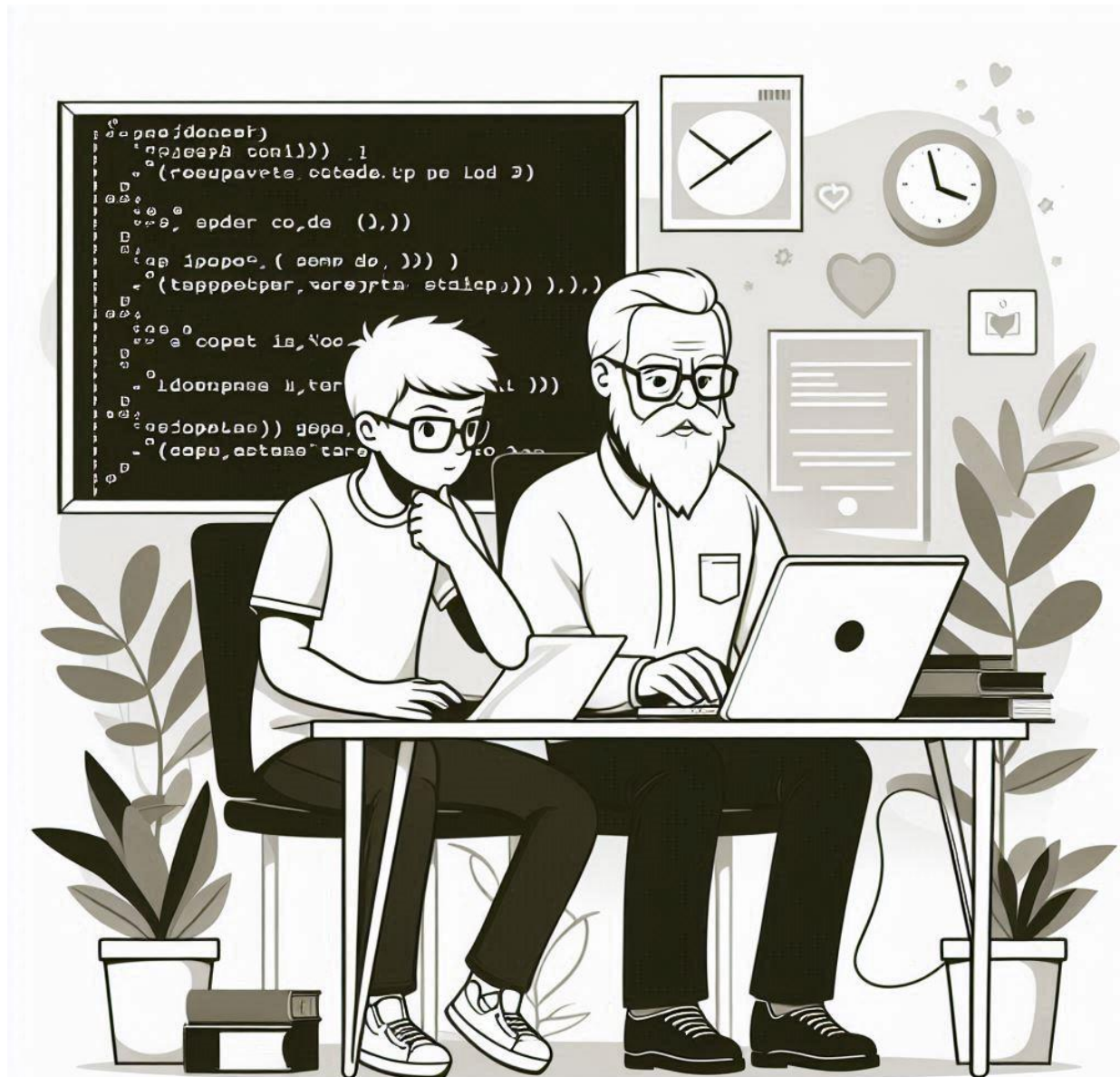
“Ask yourself “If I had to support this in production tomorrow, with no previous knowledge of it, how tempted would I be to send a glitter bomb to the developer who wrote it?”

**When you think like that, it really cleans up a lot of your code”
– Unknown Redditor**

Avoiding the lottery factor of 1



Knowledge Transfer

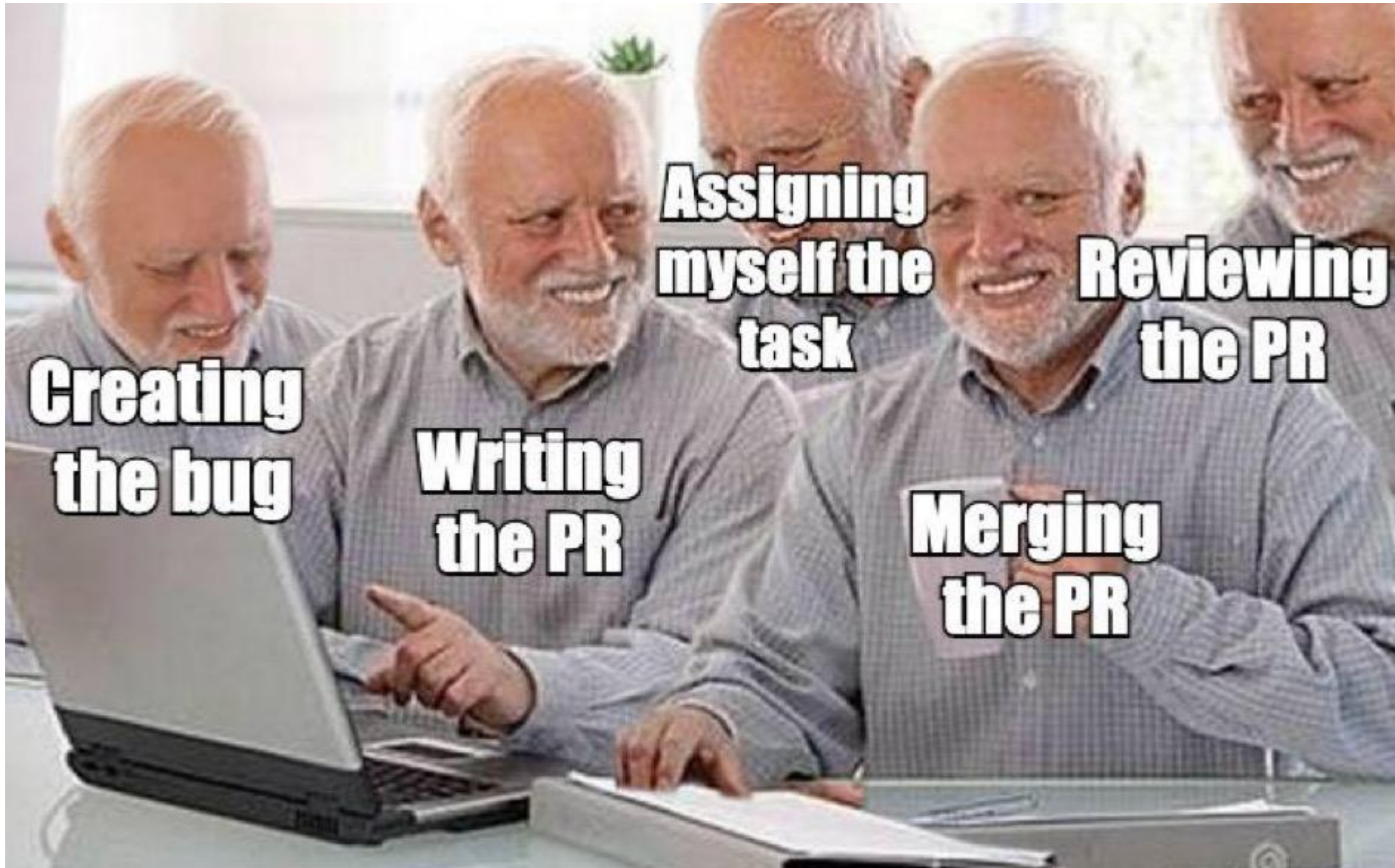


Why Code Review?

- Ensuring code quality
- Following coding guidelines
- Find bugs and edge cases
- Avoiding the lottery factor of 1
- Knowledge transfer



Why Code Review?



Review the process

Pull Requests, policies, pipelines and linters



Review the process

Pull Requests, policies, pipelines and linters

- Does it build?
- Do all the tests pass?
- Are translations done?
- Is it following the coding guidelines?



Linting use cases – Coding Conventions

You, 54 seconds ago | 1 author (You)

var

2 references

HideMessage: Boolean;

(global) HideMessage: Boolean

Global variables should be prefixed with "g" AL(TS0002)

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

```
if SkipCheck then  
    exit;
```

The exit keyword ends the procedure with the specified exit value.

EXIT keyword must be Capitalized AL(TS0005)

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

Linting use cases – Maintainability improvements

1 reference | 0% Coverage

```
local procedure ImportVendorBankAccounts()  
begin  
    ImportBankAccountsFromExcel();  
  
    Commit();  
  
    AssignPrimaryBankAccount();  
end;
```

1 reference | 0% Coverage

```
local procedure CreateNewCustomer(Number: Code[20]; Name: Text[100])  
var  
    Customer: Record Customer;  
begin  
    Customer.Init();  
    Customer."No." := Number;  
    Customer.Name := Name;  
    Customer.Insert();  
end;
```

```
procedure Commit()
```

Ends the current write transaction.

Commit() needs a comment to justify its existence. Either a leading or a trailing comment. AL(TS0012)

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

```
procedure Insert(): Boolean
```

Inserts a record into a table without executing the code in the OnInsert trigger.

Internal Methods must be invoked with explicit parameters AL(TS0013)

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

Linting use cases

```
table tableextension 50100 CustomerExt extends Customer
```

```
{
    {
        fields
        {
            {
                0 references
                field(50100; IsBroker; Boolean)
                {
                    Caption = 'Is Broker';
                    DataClassification = CustomerContent;
                }
            }
            {
                0 references
                field(50101; Occupation; Text[250])
                {
                    Caption = 'Occupation';
                    DataClassification = CustomerContent;
                }
            }
            {
                0 references
                field(50102; LegislationOffice; Text[50])
                {
                    Caption = 'Legislation Office';
                    DataClassification = CustomerContent;
                }
            }
            {
                0 references
                field(50103; StrictConfidentiality; Boolean)
                {
                    Caption = 'Strict Confidentiality';
                    DataClassification = CustomerContent;
                }
            }
        }
    }
}
```

Conflicting ID, Name or Type with Table 'Contact' AL(CM0027)

(field) Occupation: Text[250]

View Problem (Alt+F8) Quick Fix... (Ctrl+.)

field(50101; Occupation; Text[250]) Conflicting ID, Name or Type with Table 'Contact' AL(CM0027)

Caption = 'Occupation';

DataClassification = CustomerContent;

```
tableextension 50101 ContactExt extends Contact
```

```
{
    {
        fields
        {
            {
                0 references
                field(50100; HasValidCertification; Boolean)
                {
                    Caption = 'Has Valid Certification';
                    DataClassification = CustomerContent;
                }
            }
            {
                0 references
                field(50101; IsBroker; Boolean)
                {
                    Caption = 'Is Broker';
                    DataClassification = CustomerContent;
                }
            }
            {
                0 references
                field(50102; StrictConfidentiality; Boolean)
                {
                    Caption = 'Strict Confidentiality';
                    DataClassification = CustomerContent;
                }
            }
        }
    }
}
```



Review the process

Pull Requests, policies, pipelines and linters

- Does it build?
- Do all the tests pass?
- Are translations done?
- Is it following the coding guidelines?



Review the process

Size, time and frequency of pull requests

- Many smaller pull requests are better than one huge
- Make small pull requests
- No, seriously, keep pull requests small



Small work item – Small Pull Request

“But when the code's finished is too late for deep feedback.
Any pushback breeds resentment. What do you mean you
don't like my beautiful baby!?”

– Swizec Teller

<https://swizec.com/>

Small work item – Small Pull Request



Swizec Teller  @Swizec · 1d

You know wha worse than a meeting?
building the wrong thing for a week



Review the process

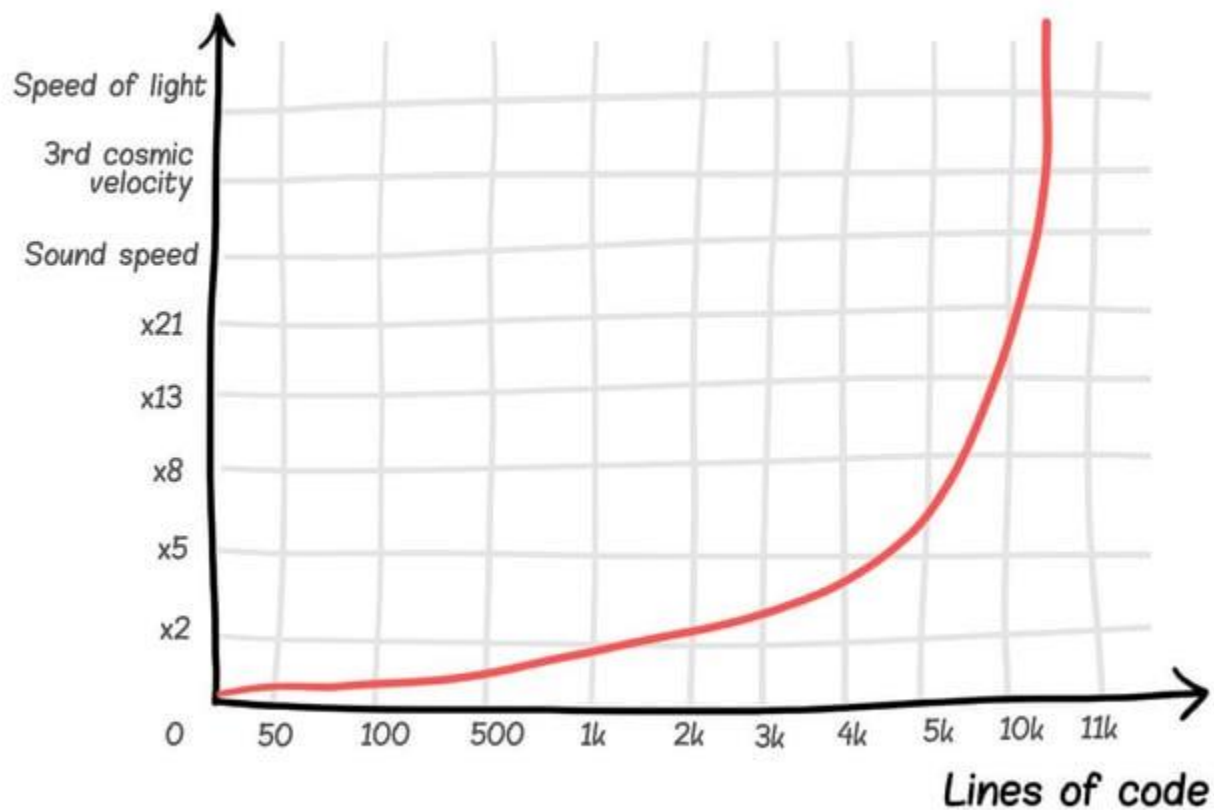
Size, time and frequency of pull requests

- Many smaller pull requests are better than one huge
 - Make small pull requests
 - No, seriously, keep pull requests small
-
- Small work items will result in a small PR.
 - Ideal work span is up to 3 days, 5 is okay
 - 1 hour is too little, 2 weeks are questionable, more than two weeks should never happen.
-
- If you're doing big PRs, plan time for code review



Make it small!

Code Review Scrolling Speed



Make it small!



Ryan Peterman ✓
@ryanlpeterman

Ask an engineer to review a 15 lines of code and they'll find plenty of issues.

Ask them to review 1000 lines and they'll say it looks good.

44 Retweets **661** Likes

Reviewing the Reviewer

- Be timely – Prioritize review work to unblock others
- Review the changes not just the syntax
- Stick to the scope



Review

Coding guide

- Different
- The ultim

Code formatting

Code should follow the AL formatting rules enforced by VS Code. No individual formatting rules should be set because different formatting ends up with unnecessary changes in each commit.

In general, there is no restriction on the line length, but lengthy lines can make the code unreadable. We recommend having a max length of 120 symbols to keep code easily scannable and readable.

When considering the issue of excessively long procedure signatures or invocations, it becomes imperative to maintain code readability and conformity to established coding standards. In cases where a procedure signature or invocation spans an excessive length, it is recommended to break the parameters into separate lines to enhance code clarity. Proper indentation plays a pivotal role in achieving this objective. An exemplary illustration of correct indentation involves aligning the parameters in a manner that facilitates quick comprehension, typically by placing each parameter on a new line and indenting them appropriately, as demonstrated below:

```
internal procedure SetAddress(  
    Address: Text[100];  
    Address2: Text[50];  
    PostCode: Code[20];  
    City: Text[30];  
    County: Text[30];  
    CountryCode: Code[10];  
    Contact: Text[100]  
)  
begin  
end;
```

```
trigger OnRun()  
begin  
    SetAddress(  
        Address,  
        Address2,  
        PostCode,  
        City,  
        County,  
        CountryCode,  
        Contact  
    )  
end;
```

<https://algu>



A Wild West of coding

```
[Scope('OnPrem')]  
9 references | 0% Coverage  
procedure VoorraadboekingsgroepGebruiken(f_Voorraadgroep: Code[20]): Boolean  
var  
    VoorraadgroepenPerBedrijfLrec: Record "Voorraadgroepen per bedrijf";  
    IText001: Label 'U heeft geen recht om dit artikel met verkoopboekingsgroep: %1 te gebruiken in deze vestiging!';  
begin  
    //INDEX 3  
    if VoorraadgroepenPerBedrijfLrec.FindFirst then // controle bestand gevuld en functionaliteit wordt gebruikt  
        if f_Voorraadgroep <> '' then  
            exit(VoorraadgroepenPerBedrijfLrec.Get(f_Voorraadgroep))  
        else  
            exit(false);  
    exit(true);  
end;
```


How will you treat “this”?

```
codeunit 50100 WhatIsThis implements ISomethingInteresting
{
    0 references
    trigger OnRun()
    var
        What: Codeunit What;
    begin
        What.Is(this);
        this.IsComingSoon();
    end;
}
```

Reviewing the Reviewer

Clear communication is key

Five types of code to look for:

- Good code
- Could-be-better code
- Bad code
- “Meh” code
- Amazing code

Adopt the power of “nit”



Review the Reviewer



Review the Reviewee

- Focus on design before code



Review the Reviewee



Thinking
Before
Coding



Coding
Before
Thinking

Review the Reviewee

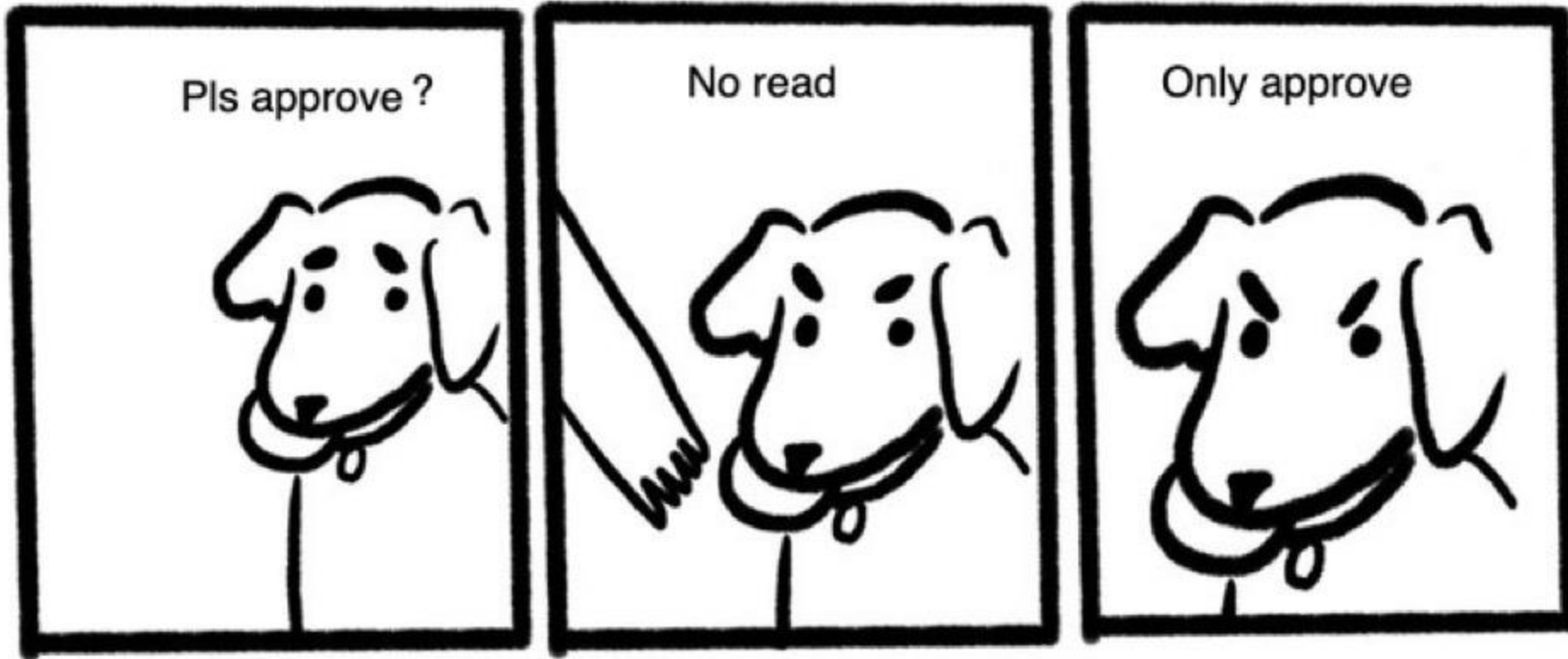
- Focus on design first
- Review yourself
- Be proactive in clarifying unclear parts
- Be open to feedback



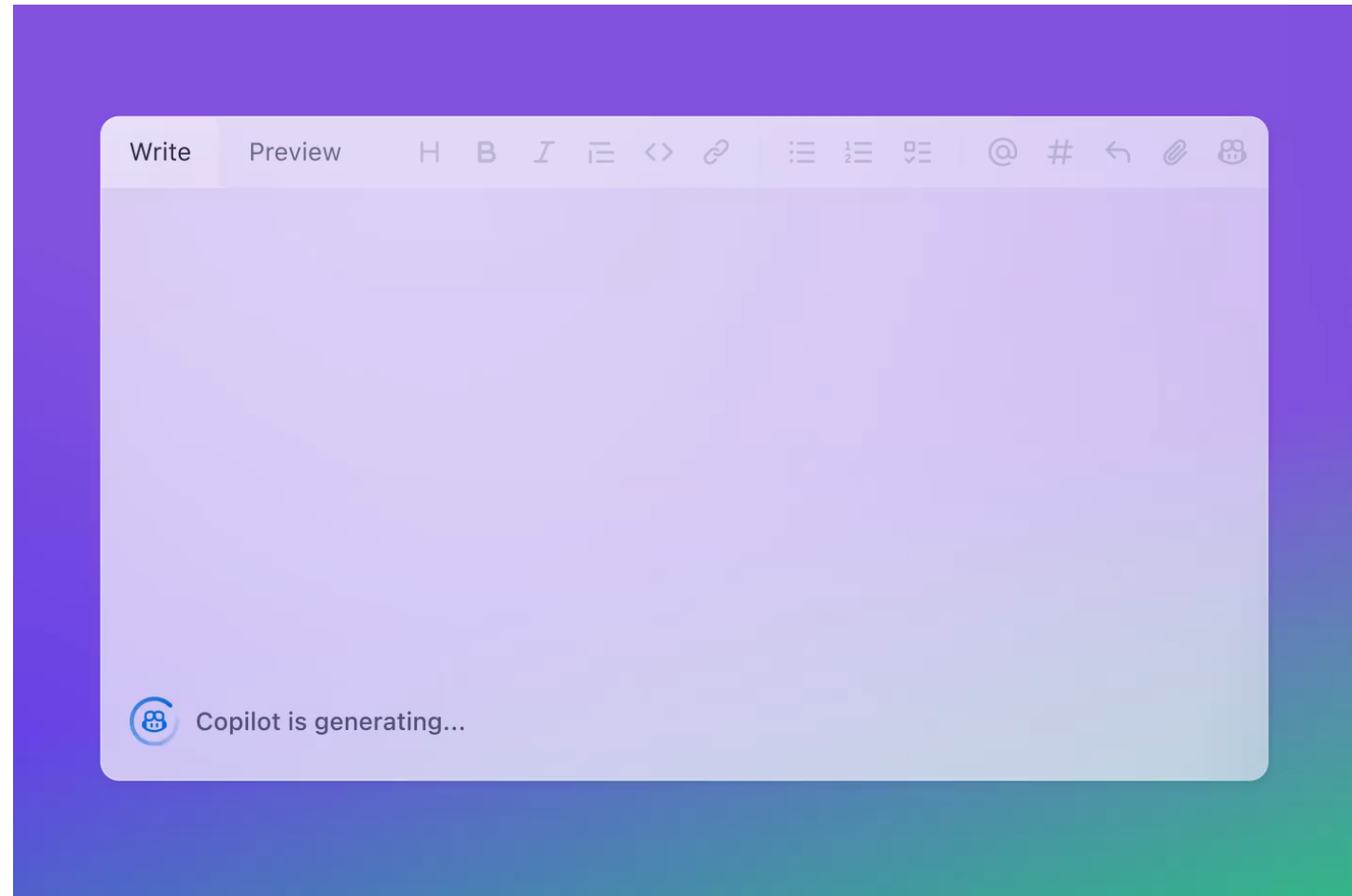
Review the Reviewee

“..I need to push this PR fast..”

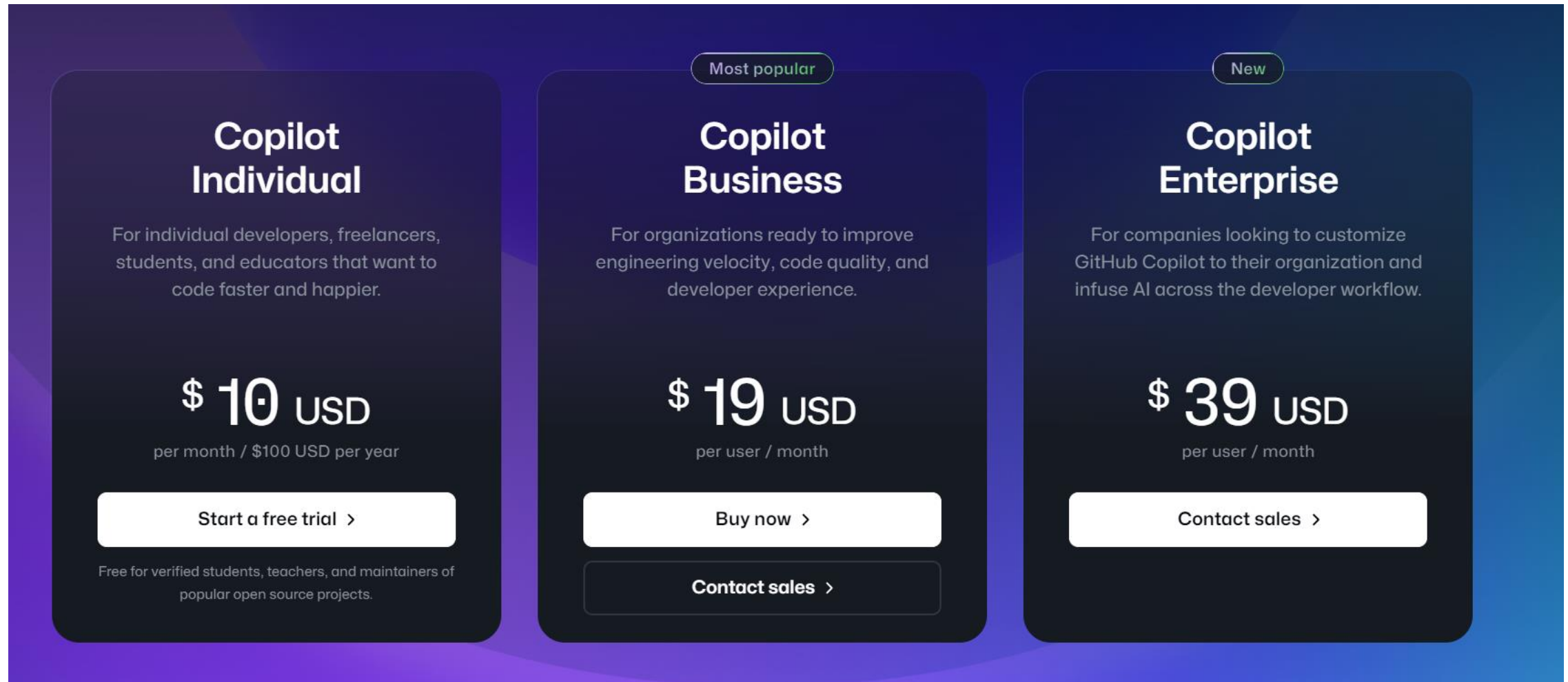
“..I don't have time to argue..”



GitHub Copilot



GitHub Copilot



The screenshot displays the GitHub Copilot pricing page with a dark blue background and three pricing cards. The 'Copilot Business' card is highlighted with a 'Most popular' badge, and the 'Copilot Enterprise' card is marked as 'New'. Each card includes a description of the target audience, the price in USD, the billing cycle, and a call-to-action button. The 'Copilot Individual' card also includes a note about being free for verified students and teachers.

Plan	Target Audience	Price (USD)	Billing Cycle	Call to Action
Copilot Individual	For individual developers, freelancers, students, and educators that want to code faster and happier.	\$10	per month / \$100 USD per year	Start a free trial >
Copilot Business	For organizations ready to improve engineering velocity, code quality, and developer experience.	\$19	per user / month	Buy now > Contact sales >
Copilot Enterprise	For companies looking to customize GitHub Copilot to their organization and infuse AI across the developer workflow.	\$39	per user / month	Contact sales >

GitHub Copilot

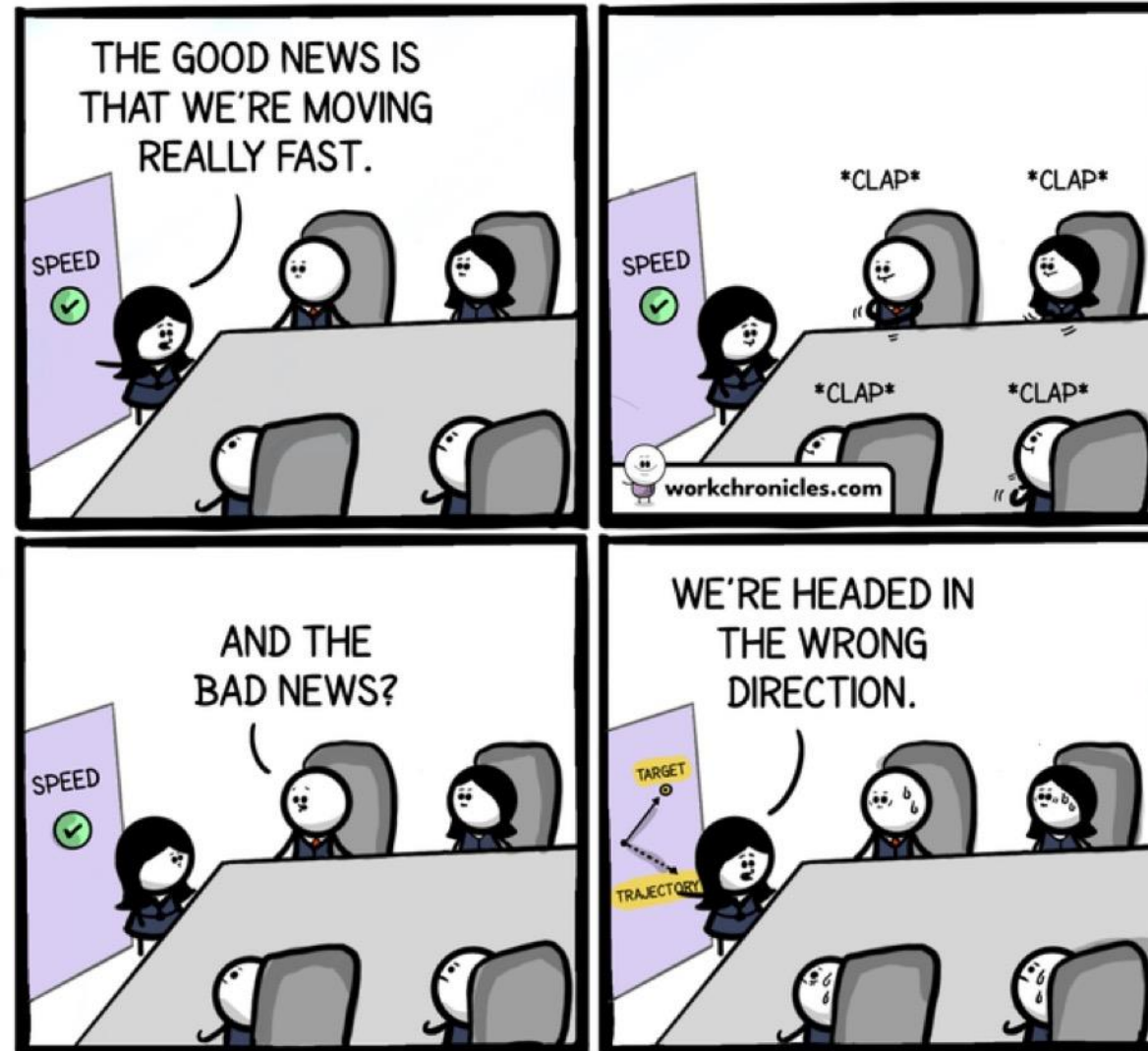
RECOMMENDED		
Free The basics for individuals and organizations \$0 per user/month Create a free organization	Team Advanced collaboration for individuals and organizations \$4 per user/month Continue with Team ▾	Enterprise Security, compliance, and flexible deployment \$21 per user/month Start a free trial Contact Sales

Key Takeaways

- Do the code review



Focus on design before code



Comics about work. Made with ❤️ & lots of coffee.
Get the comics straight to your Inbox. Join the Newsletter.

Work Chronicles
workchronicles.com

Keep pull requests small

Friday 5PM: "hey can you review this real quick, fixed a couple things"



Key Takeaways

- Do the code review
- Focus on design before code
- Keep the pull requests small
- Adopt process improvements to make reviews faster



Clearly communicate your intentions



Key Takeaways

- Do the code review
- Focus on design before code
- Keep the pull requests small
- Adopt process improvements to make reviews faster
- Clearly communicate your intention



Key Takeaways

“We need to understand that PRs aren’t an obstacle that we need to overcome, but a tool to collectively have better code.”

– James Pearson

Q&A

Any Questions?

Thank
You

